

جلسه اول



بخش ۲: انواع اشیا در R؛ بردارها و لیست‌ها

در تحلیل داده‌های پژوهشی، تقریباً هیچ‌گاه با یک مقدار منفرد سروکار نداریم. داده‌های واقعی معمولاً شامل مجموعه‌ای از مقادیر هستند؛ برای مثال وزن چندین ماهی، طول چندین میگو، غلظت اکسیژن محلول در چند ایستگاه، یا نتایج چند تیمار آزمایشی. در زبان R، پایه‌ای‌ترین ساختار برای نگهداری مجموعه‌ای از مقادیر هم‌نوع، بردار است.



مفهوم شیء در R

در R تقریباً هر چیزی که می‌سازید یک شیء است. متغیر عددی، متن، بردار، لیست، جدول داده، مدل آماری و حتی خروجی یک آزمون آماری، همگی شیء محسوب می‌شوند.

برای مثال:

```
final_weight <- 18.6
species <- "Litopenaeus vannamei"
survived <- TRUE
```

در اینجا سه شیء ساخته شده است:

- یک شیء عددی برای وزن نهایی
- یک شیء متنی برای نام گونه
- یک شیء منطقی برای وضعیت بقا

با اجرای کد زیر می‌توان اشیای موجود در محیط کاری را مشاهده کرد:

```
ls()
```

برای حذف یک شیء از محیط کاری:

```
rm(final_weight)
```

در پژوهش‌های واقعی، مدیریت اشیا اهمیت زیادی دارد؛ زیرا ممکن است در طول یک تحلیل، چندین نسخه از داده‌ها، خروجی‌ها و مدل‌ها در محیط R ساخته شوند.



بردار چیست؟

بردار ساده‌ترین و مهم‌ترین ساختار داده در R است. یک بردار مجموعه‌ای از مقادیر هم‌نوع است؛ یعنی همه عناصر آن باید از یک نوع داده باشند.

نمونه‌هایی از بردار در پژوهش‌های شیلاتی:

- وزن ۱۰ عدد ماهی

- طول کل چند نمونه ماهی
- شوری آب در چند ایستگاه
- دمای آب در روزهای مختلف
- نام چند گونه آبزی
- وضعیت زنده‌مانی چند نمونه

برای مثال، وزن پنج میگو در پایان دوره پرورش را می‌توان به‌صورت یک بردار عددی ذخیره کرد:

```
shrimp_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)
shrimp_weight
```

خروجی:

```
[1] 15.2 16.8 14.9 17.1 16.3
```

تابع `c()` برای ترکیب چند مقدار و ساخت یک بردار استفاده می‌شود. نام این تابع از واژه `combine` گرفته شده است.

ساخت بردار با تابع `c()`

رایج‌ترین روش ساخت بردار در R استفاده از تابع `c()` است.

بردار عددی

```
fish_length <- c(12.5, 14.3, 13.8, 15.1, 16.0)
fish_length
```

بررسی نوع داده:

```
class(fish_length)
```

خروجی:

```
[1] "numeric"
```

در این مثال، طول پنج ماهی در یک بردار عددی ذخیره شده است.

بردار متنی

```
species <- c("Cyprinus carpio", "Oreochromis niloticus", "Oncorhynchus mykiss")
```

```
species
```

بررسی نوع داده:

```
class(species)
```

خروجی:

```
[1] "character"
```

بردار متنی برای نگهداری نام گونه‌ها، ایستگاه‌ها، تیمارها یا کدهای نمونه مناسب است.

بردار منطقی

```
survival_status <- c(TRUE, TRUE, FALSE, TRUE, FALSE)
survival_status
```

بررسی نوع داده:

```
class(survival_status)
```

خروجی:

```
[1] "logical"
```

این بردار می‌تواند وضعیت زنده‌مانی چند نمونه را نشان دهد.

بردار طبقه‌ای

در بسیاری از مطالعات، متغیرهایی مانند تیمار، جنسیت، نوع سیستم پرورش یا فصل نمونه‌برداری ماهیت طبقه‌ای دارند.

```
diet <- factor(c("Control", "Probiotic", "Control", "Probiotic", "Synbiotic"))
diet
```

مشاهده سطوح:

```
levels(diet)
```

خروجی:

```
[1] "Control" "Probiotic" "Synbiotic"
```

اگر ترتیب سطوح اهمیت داشته باشد، بهتر است آن را به صورت صریح مشخص کنیم.

```
protein_level <- factor(c("Low", "Medium", "High", "Medium", "Low"),  
                        levels = c("Low", "Medium", "High"))  
protein_level
```



نکته مهم درباره هم‌نوع بودن عناصر بردار

در R همه عناصر یک بردار باید از یک نوع باشند. اگر در یک بردار، مقدار عددی و متنی را با هم ترکیب کنید، R معمولاً همه را به متن تبدیل می‌کند.

```
mixed_vector <- c(12.5, "Control", 15.8)  
mixed_vector  
class(mixed_vector)
```

خروجی:

```
[1] "12.5"      "Control" "15.8"  
[1] "character"
```

این رفتار در تحلیل داده‌ها می‌تواند باعث خطای جدی شود؛ زیرا مقادیر عددی ممکن است ناخواسته به متن تبدیل شوند. برای داده‌های پژوهشی، بهتر است هر ستون یا هر بردار فقط یک نوع داده داشته باشد.

ساخت بردار با تابع seq()

تابع seq() برای ساخت دنباله عددی استفاده می‌شود. این تابع در طراحی زمان‌های نمونه‌برداری، سطوح تیمار، شماره تکرارها و بازه‌های دمایی بسیار کاربرد دارد.

ساخت دنباله ساده

```
days <- seq(0, 60, by = 15)  
days
```

خروجی:

```
[1] 0 15 30 45 60
```

این بردار می‌تواند روزهای نمونه‌برداری در یک آزمایش رشد میگو را نشان دهد.

تعیین تعداد عناصر دنباله

```
salinity_gradient <- seq(10, 30, length.out = 5)
salinity_gradient
```

خروجی:

```
[1] 10 15 20 25 30
```

این بردار می‌تواند پنج سطح شوری برای یک آزمایش تحمل شوری باشد.

دنباله کاهشی

```
temperature_levels <- seq(32, 24, by = -2)
temperature_levels
```

خروجی:

```
[1] 32 30 28 26 24
```

ساخت بردار با تابع rep()

تابع rep() برای تکرار مقدار یا بردار استفاده می‌شود. این تابع در طراحی آزمایش‌ها و ساخت متغیرهای گروهی بسیار پرکاربرد است.

تکرار یک مقدار

```
rep("Control", 5)
```

خروجی:

```
[1] "Control" "Control" "Control" "Control" "Control"
```

تکرار چند مقدار

```
treatment <- rep(c("Control", "Probiotic", "Synbiotic"), times = 3)
```

```
treatment
```

خروجی:

```
[1] "Control" "Probiotic" "Synbiotic" "Control" "Probiotic" "Synbiotic"
[7] "Control" "Probiotic" "Synbiotic"
```

تکرار هر مقدار پشت سر هم

```
treatment <- rep(c("Control", "Probiotic", "Synbiotic"), each = 3)
treatment
```

خروجی:

```
[1] "Control" "Control" "Control" "Probiotic" "Probiotic" "Probiotic"
[7] "Synbiotic" "Synbiotic" "Synbiotic"
```

این روش برای ساخت تیمارها در آزمایش‌های دارای تکرار بسیار مناسب است.

ساخت شماره تکرارها

```
replicate_id <- rep(1:3, times = 3)
replicate_id
```

خروجی:

```
[1] 1 2 3 1 2 3 1 2 3
```

در طراحی آزمایش، می‌توان از ترکیب `rep()` و `factor()` برای ساخت متغیرهای طبقه‌ای استفاده کرد.

```
treatment <- factor(rep(c("Control", "Probiotic", "Synbiotic"), each = 3))
replicate_id <- rep(1:3, times = 3)

treatment
replicate_id
```

ساخت بردار با تابع `scan()`

تابع `scan()` برای ورود سریع داده‌ها به R استفاده می‌شود. این تابع به‌ویژه زمانی مفید است که پژوهشگر بخواهد چند مقدار عددی را سریع از صفحه‌کلید یا از یک فایل ساده وارد کند.

ورود داده از صفحه‌کلید

```
weights <- scan()
```

پس از اجرای دستور، R منتظر ورود داده می‌ماند. می‌توان مقادیر را وارد کرد و در پایان با یک خط خالی ورود داده را متوقف نمود.

نمونه داده قابل ورود:

```
12.4 13.1 14.8 15.0 13.9
```

پس از پایان ورود داده:

```
weights
```

ورود داده متنی با `scan()`

برای داده متنی باید نوع داده مشخص شود.

```
sites <- scan(what = character())
```

نمونه داده قابل ورود:

```
Station_A Station_B Station_C
```

خواندن داده از فایل متنی

اگر فایل متنی ساده‌ای شامل وزن‌ها وجود داشته باشد، می‌توان آن را با `scan()` خواند.

```
weights <- scan("dataset/weights.txt")
weights
```

اگر داده‌ها متنی باشند:

```
Station <- scan("dataset/Station.txt", what = character())
Station
```

تابع `scan()` برای داده‌های ساده مناسب است. برای داده‌های جدولی کامل، در بخش‌های بعدی از توابع تخصصی‌تر ورود داده استفاده خواهد شد.



اسلایس کردن بردارها

اسلایس کردن یعنی انتخاب یک یا چند عنصر از بردار. در R این کار با استفاده از کروشه انجام می‌شود.

```
shrimp_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)
```

انتخاب یک عنصر از بردار

در R اندیس‌گذاری از عدد ۱ شروع می‌شود.

```
shrimp_weight[1]
```

خروجی:

```
[1] 15.2
```

انتخاب عنصر سوم:

```
shrimp_weight[3]
```

خروجی:

```
[1] 14.9
```

انتخاب چند عنصر

```
shrimp_weight[c(1, 3, 5)]
```

خروجی:

```
[1] 15.2 14.9 16.3
```

انتخاب عناصر دوم تا چهارم:


```
shrimp_weight[2:4]
```

خروجی:

```
[1] 16.8 14.9 17.1
```

حذف یک یا چند عنصر

برای حذف عنصر از علامت منفی استفاده می‌شود.

```
shrimp_weight[-1]
```

خروجی:

```
[1] 16.8 14.9 17.1 16.3
```

حذف عناصر دوم و چهارم:

```
shrimp_weight[-c(2, 4)]
```

خروجی:

```
[1] 15.2 14.9 16.3
```

اسلایس کردن با شرط

یکی از قابلیت‌های مهم R، انتخاب عناصر برادر بر اساس شرط است.

```
shrimp_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)
```

```
shrimp_weight > 16
```

خروجی:

```
[1] FALSE TRUE FALSE TRUE TRUE
```

این خروجی یک بردار منطقی است. حال می‌توان از همین بردار منطقی برای انتخاب مقادیر استفاده کرد.

```
shrimp_weight[shrimp_weight > 16]
```

خروجی:

```
[1] 16.8 17.1 16.3
```

این روش در تحلیل داده‌های پژوهشی بسیار کاربرد دارد؛ مثلاً انتخاب ماهی‌هایی با وزن بالاتر از حد مشخص، انتخاب ایستگاه‌هایی با اکسیژن پایین، یا انتخاب نمونه‌هایی با طول بیشتر از مقدار آستانه.

مثال‌های شرطی در داده‌های شیلاتی انتخاب دماهای بالاتر از حد مطلوب

```
temperature <- c(27.5, 28.1, 31.2, 29.4, 32.0, 26.8)
```

```
high_temperature <- temperature[temperature > 30]  
high_temperature
```

انتخاب شوری‌های کمتر از مقدار بحرانی

```
salinity <- c(12, 18, 25, 30, 8, 15)
```

```
low_salinity <- salinity[salinity < 10]  
low_salinity
```

انتخاب وزن‌های در محدوده مطلوب

```
fish_weight <- c(85, 92, 110, 76, 98, 120)
```

```
selected_weight <- fish_weight[fish_weight >= 90 & fish_weight <= 110]  
selected_weight
```

نام‌گذاری عناصر بردار

گاهی لازم است هر عنصر بردار دارای نام باشد؛ مثلاً میانگین اکسیژن محلول در چند ایستگاه نمونه‌برداری.

```
dissolved_oxygen <- c(5.8, 4.9, 6.2, 5.1)
names(dissolved_oxygen) <- c("Station_A", "Station_B", "Station_C",
"Station_D")
dissolved_oxygen
```

خروجی:

```
Station_A Station_B Station_C Station_D
      5.8      4.9      6.2      5.1
```

انتخاب مقدار مربوط به یک ایستگاه:

```
dissolved_oxygen["Station_B"]
```

خروجی:

```
Station_B
      4.9
```

نام گذاری عناصر برای داده های کوچک و خلاصه شده مفید است، اما برای داده های جدولی بزرگ بهتر است از ساختارهایی مانند data frame استفاده شود که در بخش های بعدی بررسی خواهد شد.

توابع مهم با ورودی برداری

بسیاری از توابع پایه R به صورت طبیعی با بردار کار می کنند. یعنی اگر یک بردار به آن ها بدهید، محاسبه را روی کل بردار انجام می دهند.

محاسبه طول بردار

```
shrimp_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)

length(shrimp_weight)
```

خروجی:

```
[1] 5
```

تابع `length()` تعداد عناصر بردار را نشان می دهد.

محاسبه مجموع

```
sum(shrimp_weight)
```

خروجی:

```
[1] 80.3
```

محاسبه میانگین

```
mean(shrimp_weight)
```

خروجی:

```
[1] 16.06
```

محاسبه میانه

```
median(shrimp_weight)
```

خروجی:

```
[1] 16.3
```

محاسبه کمینه و بیشینه

```
min(shrimp_weight)  
max(shrimp_weight)
```

خروجی:

```
[1] 14.9
```

```
[1] 17.1
```

محاسبه دامنه تغییرات

```
range(shrimp_weight)
```

خروجی:

```
[1] 14.9 17.1
```

محاسبه واریانس و انحراف معیار

```
var(shrimp_weight)  
sd(shrimp_weight)
```

این توابع برای توصیف پراکندگی وزن، طول، شوری، دما یا هر متغیر کمی دیگر کاربرد دارند. واریانس نمونه‌ای در R به صورت زیر محاسبه می‌شود:

$$= \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

انحراف معیار نیز برابر است با:

=

خلاصه آماری سریع

```
summary(shrimp_weight)
```

خروجی:

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
14.90	15.20	16.30	16.06	16.80	17.10

تابع `summary()` برای مرور سریع توزیع یک متغیر عددی بسیار مفید است.

⚠ کار با مقادیر گم‌شده در بردارها

در داده‌های پژوهشی، وجود مقدار گم‌شده رایج است. ممکن است وزن یک نمونه ثبت نشده باشد، سنسور دما در یک روز خطا داده باشد، یا داده اکسیژن محلول برای یک ایستگاه موجود نباشد.

در R مقدار گم‌شده با `NA` نمایش داده می‌شود.

```
shrimp_weight <- c(15.2, 16.8, NA, 17.1, 16.3)
shrimp_weight
```

اگر روی برداری که دارای NA است میانگین بگیریم:

```
mean(shrimp_weight)
```

خروجی:

```
[1] NA
```

برای حذف مقادیر گمشده از محاسبه:

```
mean(shrimp_weight, na.rm = TRUE)
```

تابع‌های زیر نیز آرگومان `na.rm` را می‌پذیرند:

```
sum(shrimp_weight, na.rm = TRUE)
min(shrimp_weight, na.rm = TRUE)
max(shrimp_weight, na.rm = TRUE)
sd(shrimp_weight, na.rm = TRUE)
```

برای بررسی وجود مقدار گمشده:

```
is.na(shrimp_weight)
```

برای انتخاب مقادیر غیر گمشده:

```
shrimp_weight[!is.na(shrimp_weight)]
```



محاسبات برداری در R

یکی از ویژگی‌های قدرتمند R، انجام محاسبات مستقیم روی بردارهاست. اگر یک عمل ریاضی را روی بردار انجام دهید، R آن را روی تمام عناصر بردار اعمال می‌کند.

```
initial_weight <- c(2.1, 2.3, 2.0, 2.4, 2.2)
final_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)

weight_gain <- final_weight - initial_weight
weight_gain
```

```
[1] 13.1 14.5 12.9 14.7 14.1
```

محاسبه نرخ رشد ویژه برای چند تکرار:

```
days <- 60

sgr <- (log(final_weight) - log(initial_weight)) / days * 100
sgr
```

فرمول نرخ رشد ویژه:

$$SGR(\%) = \frac{\ln(W_f) - \ln(W_i)}{t} \times 100$$

محاسبه درصد افزایش وزن:

```
weight_gain_percent <- (final_weight - initial_weight) / initial_weight * 100
weight_gain_percent
```

فرمول درصد افزایش وزن:

$$WG(\%) = \frac{W_f - W_i}{W_i} \times 100$$

مقایسه برداری و تولید خروجی منطقی

در R می‌توان بردارها را با مقدار آستانه یا با بردار دیگر مقایسه کرد.

```
dissolved_oxygen <- c(5.8, 4.9, 6.2, 5.1, 3.8)
oxygen_ok <- dissolved_oxygen >= 5
oxygen_ok
```

خروجی:

```
[1] TRUE FALSE TRUE TRUE FALSE
```

محاسبه تعداد ایستگاه‌های دارای اکسیژن مناسب:

```
sum(oxygen_ok)
```

خروجی:

```
[1] 3
```

در R مقدار TRUE در محاسبات عددی برابر ۱ و مقدار FALSE برابر صفر در نظر گرفته می‌شود. بنابراین `sum()` روی بردار منطقی، تعداد موارد درست را محاسبه می‌کند. محاسبه درصد ایستگاه‌های مناسب:

```
mean(oxygen_ok) * 100
```

خروجی:

```
[1] 60
```



ذخیره خروجی ساده با `cat()`

تابع `cat()` برای نمایش یا ذخیره خروجی متنی ساده استفاده می‌شود. این تابع زمانی مفید است که بخواهید نتیجه محاسبه را به شکل خوانا در Console یا فایل متنی ذخیره کنید.

نمایش خروجی در Console

```
mean_weight <- 16.06  
cat("Mean final weight:", mean_weight, "g\n")
```

خروجی:

```
Mean final weight: 16.06 g
```

علامت `n\` باعث رفتن به خط بعد می‌شود.

ذخیره خروجی در فایل متنی

```
mean_weight <- 16.06  
sd_weight <- 0.91  
  
cat("Mean final weight:", mean_weight, "g\n",  
    "Standard deviation:", sd_weight, "g\n",  
    file = "growth_summary.txt")
```

پس از اجرای کد، فایل زیر در مسیر کاری ساخته می‌شود:

افزودن خروجی جدید به فایل قبلی

اگر بخواهید خروجی جدید به انتهای فایل موجود اضافه شود، از `append = TRUE` استفاده می‌شود.

```
survival_rate <- 94

cat("Survival rate:", survival_rate, "%\n",
    file = "growth_summary.txt",
    append = TRUE)
```

ذخیره خلاصه محاسبات رشد

```
initial_weight <- c(2.1, 2.3, 2.0, 2.4, 2.2)
final_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)
days <- 60

weight_gain <- final_weight - initial_weight
sgr <- (log(final_weight) - log(initial_weight)) / days * 100

cat("Mean initial weight:", mean(initial_weight), "g\n",
    "Mean final weight:", mean(final_weight), "g\n",
    "Mean weight gain:", mean(weight_gain), "g\n",
    "Mean SGR:", mean(sgr), "% per day\n",
    file = "shrimp_growth_summary.txt")
```

در گزارش‌نویسی حرفه‌ای‌تر، از ابزارهایی مانند R Markdown یا Quarto استفاده می‌شود، اما `cat()` برای ذخیره خروجی‌های ساده و سریع مناسب است.

جدول توابع متداول برای کار با بردارها

تابع	کاربرد	نمونه
<code>c()</code>	ساخت بردار از چند مقدار	<code>c(12.5, 14.2, 13.8)</code>
<code>seq()</code>	ساخت دنباله عددی	<code>seq(0, 60, by = 15)</code>
<code>rep()</code>	تکرار مقدار یا بردار	<code>rep("Control", 3)</code>
<code>scan()</code>	ورود سریع داده از صفحه‌کلید یا فایل	<code>scan()</code>
<code>length()</code>	محاسبه تعداد عناصر بردار	<code>length(weight)</code>

تابع	کاربرد	نمونه
<code>names()</code>	نام گذاری یا مشاهده نام عناصر بردار	<code>names(oxygen) <- c("A", "B")</code>
<code>sum()</code>	محاسبه مجموع عناصر	<code>sum(weight)</code>
<code>mean()</code>	محاسبه میانگین	<code>mean(weight)</code>
<code>median()</code>	محاسبه میانه	<code>median(weight)</code>
<code>min()</code>	محاسبه کمینه	<code>min(weight)</code>
<code>max()</code>	محاسبه بیشینه	<code>max(weight)</code>
<code>range()</code>	محاسبه کمینه و بیشینه	<code>range(weight)</code>
<code>var()</code>	محاسبه واریانس نمونه‌ای	<code>var(weight)</code>
<code>sd()</code>	محاسبه انحراف معیار نمونه‌ای	<code>sd(weight)</code>
<code>summary()</code>	خلاصه آماری سریع	<code>summary(weight)</code>
<code>is.na()</code>	بررسی مقادیر گمشده	<code>is.na(weight)</code>
<code>sort()</code>	مرتب‌سازی عناصر	<code>sort(weight)</code>
<code>unique()</code>	استخراج مقادیر یکتا	<code>unique(treatment)</code>
<code>cat()</code>	نمایش یا ذخیره خروجی متنی ساده	<code>cat("Mean:", mean_weight)</code>



لیست چیست؟

لیست در R ساختاری انعطاف پذیرتر از بردار است. برخلاف بردار که همه عناصر آن باید هم‌نوع باشند، یک لیست می‌تواند اشیای مختلف با انواع متفاوت را در کنار هم نگهداری کند.

برای مثال، در یک مطالعه رشد میگو ممکن است بخواهیم اطلاعات زیر را در یک شیء واحد ذخیره کنیم:

- نام گونه
- نام تیمار
- وزن‌های اولیه
- وزن‌های نهایی
- مدت پرورش
- درصد بقا
- توضیح پژوهشی

این اجزا همگی نوع یکسانی ندارند. برخی عددی، برخی متنی، برخی برداری و برخی طبقه‌ای هستند. بنابراین لیست گزینه مناسبی است.

ساخت لیست با `list()`

برای ساخت لیست از تابع `list()` استفاده می‌شود.

```
experiment <- list(  
  species = "Litopenaeus vannamei",  
  treatment = "Probiotic",  
  initial_weight = c(2.1, 2.3, 2.0, 2.4, 2.2),  
  final_weight = c(15.2, 16.8, 14.9, 17.1, 16.3),  
  days = 60,  
  survival_rate = 94  
)
```

experiment

در اینجا یک لیست با چند عضو ساخته شده است. هر عضو می تواند نوع متفاوتی داشته باشد.

بررسی ساختار لیست

برای مشاهده ساختار لیست از `str()` استفاده می شود.

```
str(experiment)
```

خروجی نمونه:

```
List of 6  
 $ species      : chr "Litopenaeus vannamei"  
 $ treatment    : chr "Probiotic"  
 $ initial_weight : num [1:5] 2.1 2.3 2 2.4 2.2  
 $ final_weight  : num [1:5] 15.2 16.8 14.9 17.1 16.3  
 $ days         : num 60  
 $ survival_rate : num 94
```

تابع `str()` برای بررسی سریع ساختار اشیای پیچیده، به ویژه لیست ها و خروجی مدل های آماری، بسیار مهم است.

نام گذاری اعضای لیست

بهترین روش ساخت لیست، نام گذاری اعضا هنگام ساخت آن است.

```
water_quality <- list(  
  station = c("Station_A", "Station_B", "Station_C"),  
  temperature = c(28.5, 29.1, 27.8),  
  salinity = c(18, 22, 20),  
  dissolved_oxygen = c(5.8, 4.9, 6.2)  
)
```

```
water_quality
```

اگر لیست بدون نام ساخته شده باشد، می‌توان بعداً برای اعضای آن نام تعیین کرد.

```
sample_info <- list("Pond_A", 25, 5.6)

names(sample_info) <- c("pond_id", "salinity", "dissolved_oxygen")

sample_info
```

مشاهده نام اعضا:

```
names(sample_info)
```

دسترسی به اعضای لیست

برای دسترسی به اعضای لیست چند روش وجود دارد.

دسترسی با علامت `$`

```
experiment$species
```

خروجی:

```
[1] "Litopenaeus vannamei"
```

```
experiment$final_weight
```

خروجی:

```
[1] 15.2 16.8 14.9 17.1 16.3
```

این روش زمانی استفاده می‌شود که اعضای لیست نام داشته باشند.

دسترسی با کروشه دوتایی

```
experiment[["days"]]
```

خروجی:

یا با شماره عضو:

```
experiment[[1]]
```

تفاوت گروه تکی و گروه دوتایی

گروه تکی یک زیرلیست برمی گرداند.

```
experiment["species"]
```

خروجی، همچنان یک لیست است.

گروه دوتایی خود عضو را برمی گرداند.

```
experiment[["species"]]
```

خروجی، مقدار متنی عضو است.

این تفاوت در تحلیل‌های پیشرفته‌تر اهمیت زیادی دارد؛ زیرا برخی توابع انتظار دارند ورودی آن‌ها بردار یا مقدار عددی باشد، نه زیرلیست.

افزودن عضو جدید به لیست

می‌توان پس از ساخت لیست، عضو جدید به آن اضافه کرد.

```
experiment$fcr <- 1.45
```

```
experiment
```

ضریب تبدیل غذایی یا FCR معمولاً به صورت زیر تعریف می‌شود:

$$R = \frac{ita}{W_{itai}}$$

اگر خوراک مصرفی و افزایش وزن مشخص باشد:

```
feed_intake <- 21.5
```

```
weight_gain <- 14.0
```

```
experiment$fcr <- feed_intake / weight_gain  
experiment$fcr
```

تغییر مقدار یک عضو در لیست

```
experiment$survival_rate <- 95.2  
experiment$survival_rate
```

حذف عضو از لیست

برای حذف یک عضو از لیست، مقدار آن را `NULL` قرار می‌دهیم.

```
experiment$fcr <- NULL
```

لیست‌های تو در تو

یک لیست می‌تواند درون خود لیست دیگری داشته باشد. این ساختار برای نگهداری اطلاعات پیچیده پژوهشی مناسب است.

```
study <- list(  
  metadata = list(  
    species = "Litopenaeus vannamei",  
    location = "Shrimp farm",  
    duration_days = 60  
  ),  
  growth = list(  
    initial_weight = c(2.1, 2.3, 2.0, 2.4, 2.2),  
    final_weight = c(15.2, 16.8, 14.9, 17.1, 16.3)  
  ),  
  water_quality = list(  
    temperature = c(28.5, 29.1, 27.8),  
    salinity = c(18, 22, 20),  
    dissolved_oxygen = c(5.8, 4.9, 6.2)  
  )  
)  
  
study
```

دسترسی به عضو داخلی:

```
study$metadata$species
```

دسترسی به وزن نهایی:

```
study$growth$final_weight
```

محاسبه میانگین وزن نهایی از داخل لیست:

```
mean(study$growth$final_weight)
```

تفاوت کاربردی بردار و لیست در پژوهش

ویژگی	بردار	لیست
نوع عناصر	هم نوع	می تواند متفاوت باشد
مناسب برای	یک متغیر پژوهشی	مجموعه ای از اطلاعات متفاوت
مثال	وزن چند میگو	اطلاعات کامل یک آزمایش
دسترسی به عناصر	با اندیس یا شرط	با نام عضو یا اندیس
کاربرد آماری مستقیم	بسیار زیاد	بیشتر برای سازمان دهی داده ها و خروجی ها

اگر داده های شما شامل اندازه گیری های یک متغیر باشد، بردار مناسب است. اگر بخواهید چند نوع داده و خروجی را در یک شیء واحد نگهداری کنید، لیست مناسب تر است.



تمرین های خودآزمایی

تمرین ساخت بردار وزن

وزن نهایی ۸ عدد میگو را در یک بردار به نام `final_weight` ذخیره کنید. مقادیر فرضی زیر را استفاده کنید:

```
14.8, 15.2, 16.1, 13.9, 17.0, 15.7, 16.4, 14.5
```

سپس موارد زیر را محاسبه کنید:

- تعداد نمونه ها
- میانگین وزن
- کمترین وزن
- بیشترین وزن
- انحراف معیار وزن

تمرین ساخت روزهای نمونه برداری

با استفاده از `()seq` برداری بسازید که روزهای نمونه برداری از روز صفر تا روز ۷۰ را با فاصله ۱۴ روز نشان دهد.

تمرین ساخت تیمارها

با استفاده از `()rep` برای یک آزمایش دارای چهار تیمار و سه تکرار، بردار تیمار بسازید.

تیمارها:

Control
Probiotic
Prebiotic
Synbiotic

هر تیمار باید سه بار پشت سر هم تکرار شود.

تمرین اسلایس شرطی

بردار زیر اکسیژن محلول چند استخر پرورش را نشان می دهد:

```
dissolved_oxygen <- c(5.6, 4.8, 6.1, 3.9, 5.2, 4.5, 6.4)
```

موارد زیر را استخراج کنید:

- مقادیر کمتر از ۵
- مقادیر برابر یا بیشتر از ۵
- تعداد استخرهایی که اکسیژن مناسب دارند
- درصد استخرهایی که اکسیژن مناسب دارند

تمرین ساخت لیست پژوهشی

یک لیست به نام `shrimp_trial` بسازید که شامل اعضای زیر باشد:

- نام گونه
- نام تیمار
- وزن اولیه چند نمونه
- وزن نهایی چند نمونه
- مدت پرورش
- درصد بقا

سپس میانگین وزن نهایی را با دسترسی به عضو مناسب لیست محاسبه کنید.

مثال واقعی پایان بخش: سازمان‌دهی داده‌های اولیه یک آزمایش رشد میگو

در یک آزمایش رشد، پژوهشگری اثر مکمل پروبیوتیک را بر رشد میگوی وانامی بررسی کرده است. داده‌های اولیه و نهایی پنج تکرار آزمایشی به صورت زیر ثبت شده‌اند:

تکرار	وزن اولیه	وزن نهایی	تعداد اولیه	تعداد نهایی
۱	۲.۱	۱۵.۲	۱۰۰	۹۴
۲	۲.۳	۱۶.۸	۱۰۰	۹۶
۳	۲.۰	۱۴.۹	۱۰۰	۹۱
۴	۲.۴	۱۷.۱	۱۰۰	۹۵
۵	۲.۲	۱۶.۳	۱۰۰	۹۴

هدف، ذخیره داده‌ها در بردارها، محاسبه شاخص‌های اولیه رشد و بقا، سازمان‌دهی اطلاعات در یک لیست و ذخیره خلاصه نتایج در فایل متنی است.

```
species <- "Litopenaeus vannamei"
treatment <- "Probiotic"
days <- 60

replicate_id <- 1:5
initial_weight <- c(2.1, 2.3, 2.0, 2.4, 2.2)
final_weight <- c(15.2, 16.8, 14.9, 17.1, 16.3)
initial_number <- c(100, 100, 100, 100, 100)
final_number <- c(94, 96, 91, 95, 94)

weight_gain <- final_weight - initial_weight
sgr <- (log(final_weight) - log(initial_weight)) / days * 100
survival_rate <- final_number / initial_number * 100

mean_initial_weight <- mean(initial_weight)
mean_final_weight <- mean(final_weight)
mean_weight_gain <- mean(weight_gain)
mean_sgr <- mean(sgr)
mean_survival_rate <- mean(survival_rate)

shrimp_trial <- list(
  species = species,
  treatment = treatment,
  days = days,
  replicate_id = replicate_id,
  initial_weight = initial_weight,
  final_weight = final_weight,
  initial_number = initial_number,
  final_number = final_number,
```

```
weight_gain = weight_gain,  
sgr = sgr,  
survival_rate = survival_rate  
)  
  
str(shrimp_trial)  
  
cat("Species:", shrimp_trial$species, "\n",  
    "Treatment:", shrimp_trial$treatment, "\n",  
    "Culture days:", shrimp_trial$days, "\n",  
    "Mean initial weight:", mean_initial_weight, "g\n",  
    "Mean final weight:", mean_final_weight, "g\n",  
    "Mean weight gain:", mean_weight_gain, "g\n",  
    "Mean SGR:", mean_sgr, "% per day\n",  
    "Mean survival rate:", mean_survival_rate, "%\n",  
    file = "shrimp_trial_summary.txt")
```

در این مثال، داده‌های خام در قالب بردارها ذخیره شده‌اند، محاسبات اصلی به صورت برداری انجام شده است، سپس کل اطلاعات آزمایش در یک لیست منظم قرار گرفته و خلاصه نتایج در یک فایل متنی ذخیره شده است. این الگو، پایه ورود به ساختارهای جدولی و تحلیل‌های آماری در بخش‌های بعدی خواهد بود.